SIGCSE TS 2026

Student Research Competition

Visualizing and Sonifying Abstract Syntax Trees for Accessibility

***Jacob Bell**, Anna Deschamps, Eva Kapoor, Salyan Karki, Julian Kim,*
Nicole Moreno González**, William Pitchford, Elene Sturua, & **Charlotte Wade

Grinnell College

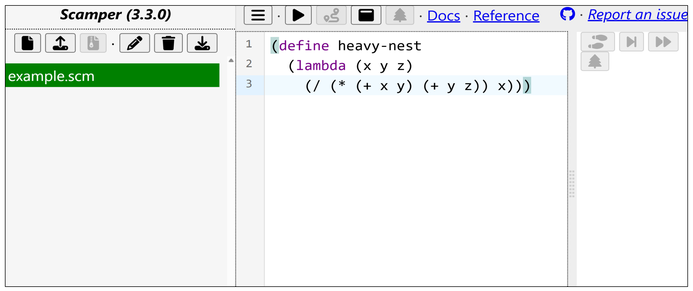
Introduction

When faculty and developers design pedagogical materials, they often focus on the “typical” student. However, students vary widely in how they learn and approach materials. Given that approximately 20% of the undergraduate population has some form of disability [1], often a “hidden” disability, tools and materials should account for the needs of such students and provide them with the opportunity to succeed in their educational program of choice. Such accessibility is often a legal requirement [2].

In this project, we extend Scamper [3], a Scheme-based IDE for teaching novice programmers, with visual parse trees, an approach that has shown some success for neurodiverse learners, particularly those who struggle with the syntax of program code when attempting to read that code. Although such visual approaches might disadvantage blind and low-vision learners, by sonifying the trees, we also make it easier for such learners to navigate both parse trees and the code.

Context/Background

Scheme is a functional programming language of the LISP family that is used in introductory computer science courses at Grinnell College. Scheme has a nested parentheses syntax with an emphasis on recursion and higher-order functions, which makes it suitable for teaching foundational computer science concepts. While this design is pedagogically strong, it can cause a variety of issues to arise. For neurodiverse users, it may hinder their ability to mentally track code structure, and blind and low-vision (BLV) learners may have difficulty navigating code using screen readers. All these reasons motivated our work on accessible Abstract Syntax Tree (AST) visualizations.



The scamper IDE and a nested function declaration

The Abstract Syntax Tree

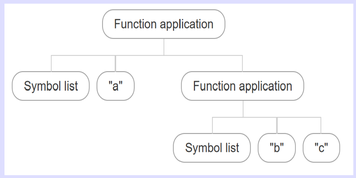
The Abstract Syntax Tree is accessible through two buttons, one providing a larger "visual" tree, and the other a more compact "textual" tree, both making explicit the tree-like structure of Scamper source code.

The AST is implemented in Typescript to communicate directly with Scamper's internal parser. It receives a copy of the parser's output, then processes that output into a syntax tree. Scamper embeds text-location data in its tokenizer, allowing us to correctly pair each tree node with its location in the original source code.

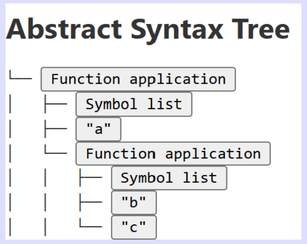
The textual tree is navigable by screen reader and is designed such that it makes sense when read aloud. We use information from Scamper's built-in parser to determine the context of expressions, allowing us to label nodes of the tree with their purpose in the code.

An example of a nested list shown in the form of an ASCII AST and a traditional tree node AST.

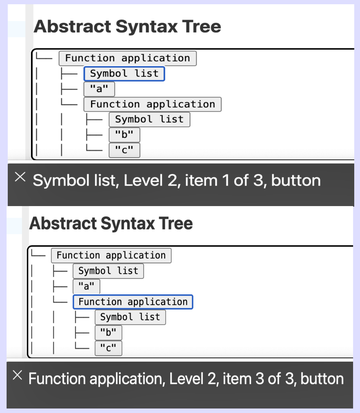
```
1  ; example.scm
2
3  (list "a" (list "b" "c"))
```



A more visual AST



A more textual AST



VoiceOver Sonification. Includes: syntax classification, nesting level, position within level.

Selecting a node of the tree with a mouse or screen reader jumps to the corresponding location in the editor view and highlights the relevant code, allowing the AST to be used as a navigational tool for BLV users.

Results and Conclusion

Our AST prototype made the Scamper code interface navigable using a keyboard alone, and it allowed screen readers to interpret interface components and code structure. By using the AST, we could more easily explore nested Scheme expressions, maintain awareness of code structure, and move between related code segments without relying on visual cues. These improvements show great potential to reduce barriers for blind and low-vision learners while also making syntactic structure more explicit for neurodiverse.

We want to highlight that accessibility is not a one-time fix, but an ongoing process that requires testing, structural support, and continued attention to the needs of all learners. While our efforts focus primarily on BLV and neurodiverse users, we hope to continue this work to support a broader range of disabilities.

Limitations

While we believe this work has many benefits, we would like to acknowledge a few limitations. Firstly, and most importantly, although we evaluated Scamper using screen readers and keyboard navigation on our own, we did not conduct formal user testing with BLV participants. Second, with the current implementation of the AST, its structure can become non-intuitive when the user code is syntactically invalid. This can lead to confusion among programmers during their early stages of programming. Additionally, the implementation is tightly integrated with Scamper’s parser. Changes to the current parser introduce the risk of breaking the AST. On that same note, adjusting the AST framework to be included in other IDEs or programming environments would be difficult without substantial modification.

References

[1] National Center for Education Statistics. 2025. Fast Facts: Students with Disabilities. https://nces.ed.gov/fastfacts/display.asp?id=60
[2] Natalie L. Shaheen. 2022. Technology accessibility: How U.S. K-12 schools are enacting policy and addressing the equity imperative. *Computers Education 179* (April 2022). doi:10.1016/j.compedu.2021.104414
[3] Peter-Michael Osera. 2025. Scamper: A mini-Scheme for teaching. https://scamper.cs.grinnell.edu